



PRODUCTO 2

NOMBRE DE LA PROFESORA:
HAYDEE GÓMEZ DÍAZ

NOMBRE DE LOS ALUMNOS:
MARIN CUAXILOA FERNANDA
HERNÁNDEZ AGUILAR BRAULIO
CRUZ SÁNCHEZ ARIEL ABRAHAM

MATRICULA:
2311082477
2311081245
2311082480

GRADO Y GRUPO:
9-D
PUEBLA, PUE

Contenido

Introducción	3
Git y GitHub	5
Justificación de su uso para versionamiento	5
Flujo de trabajo	7
Git Flow	7
Parámetros de configuración de Git y GitHub	8
Git	8
GitHub	10
Clonación del repositorio en los locales	15
Evidencia de haber colaborado todo el equipo en el repositorio	16
Enlace del repositorio en funcionamiento	17
Se presenta algunas ramas creadas en el repositorio	17
Presentan evidencia del desarrollo de su Web	18
Conclusión	20
Bibliografía	21

Introducción

En el desarrollo de software moderno, la organización, el control de los cambios y el trabajo colaborativo son aspectos fundamentales para garantizar la calidad de un proyecto. A medida que las aplicaciones se vuelven más complejas y los equipos de desarrollo aumentan en tamaño, resulta indispensable contar con herramientas y metodologías que permitan administrar el código fuente de manera eficiente. En este contexto, el control de versiones se ha convertido en una de las prácticas más importantes dentro de la ingeniería de software, ya que facilita el seguimiento de los cambios realizados en un proyecto, mejora la colaboración entre los desarrolladores y reduce el riesgo de pérdida de información.

El control de versiones es un sistema que registra todas las modificaciones realizadas sobre los archivos de un proyecto a lo largo del tiempo. Gracias a esta tecnología, es posible conocer quién realizó un cambio, cuándo lo hizo y cuál fue el motivo de la modificación. Además, permite recuperar versiones anteriores del código en caso de que ocurra algún error o se necesite regresar a un estado estable del proyecto. Esta capacidad brinda mayor seguridad y confianza durante el proceso de desarrollo, especialmente cuando varias personas trabajan de forma simultánea sobre el mismo sistema.

La importancia del versionamiento radica en que proporciona un historial completo del proyecto, permitiendo mantener un mejor control sobre su evolución. En lugar de crear múltiples copias de un mismo archivo con nombres diferentes, el sistema de control de versiones administra automáticamente todas las versiones del código, evitando confusiones y facilitando la organización. Asimismo, permite experimentar con nuevas funcionalidades sin afectar la versión principal del proyecto, ya que los desarrolladores pueden trabajar en ramas independientes y posteriormente integrar los cambios de manera segura.

Otro aspecto relevante es que el control de versiones favorece el trabajo en equipo. En proyectos colaborativos, es común que varios desarrolladores modifiquen el mismo código al mismo tiempo. Sin una herramienta adecuada, estos cambios podrían sobrescribirse o generar conflictos difíciles de resolver. Los sistemas de control de versiones permiten coordinar el trabajo de todos los integrantes, detectar conflictos cuando ocurren y ofrecer mecanismos para solucionarlos de forma controlada. Esto mejora la productividad del equipo y disminuye la posibilidad de cometer errores durante el desarrollo.

De igual manera, el flujo de trabajo desempeña un papel esencial dentro del proceso de desarrollo de software. Un flujo de trabajo bien definido establece la forma en que los desarrolladores crean nuevas funcionalidades, corrigen errores, revisan cambios e integran el código al proyecto principal. La utilización de estrategias basadas en ramas, revisiones de código y confirmaciones periódicas contribuye a mantener un desarrollo ordenado, facilita la comunicación entre los miembros del equipo y asegura que cada modificación sea evaluada antes de incorporarse al producto final.

Implementar un flujo de trabajo adecuado también permite distribuir responsabilidades entre los integrantes del equipo, mejorar la trazabilidad de los cambios y agilizar los procesos de integración y despliegue del software. Además, facilita el mantenimiento del proyecto a largo plazo, ya que cada modificación queda documentada y puede consultarse en cualquier momento. Esto resulta especialmente importante en proyectos empresariales, donde la estabilidad, la seguridad y la continuidad del desarrollo son factores críticos.

En la actualidad, herramientas como Git se han convertido en el estándar para la gestión del control de versiones, siendo utilizadas por empresas, instituciones educativas y desarrolladores de todo el mundo. Su uso permite implementar metodologías de desarrollo colaborativo y adaptarse a prácticas modernas como la integración continua y la entrega continua, mejorando significativamente la calidad y eficiencia en la creación de software.

El presente trabajo tiene como propósito aplicar los conocimientos relacionados con el control de versiones y el flujo de trabajo mediante la creación y administración de un repositorio en **GitHub**. En dicho repositorio se almacenará el proyecto desarrollado, registrando de manera ordenada cada cambio realizado a través de confirmaciones (commits), así como la utilización de ramas y la integración de modificaciones cuando sea necesario. De esta manera, el repositorio servirá como evidencia del proceso de desarrollo, demostrando la correcta implementación de las prácticas de versionamiento y la organización del trabajo colaborativo.

Git y GitHub

Justificación de su uso para versionamiento

En el desarrollo de software, la correcta administración del código fuente es un aspecto fundamental para garantizar la calidad, organización y mantenimiento de cualquier proyecto. A medida que una aplicación evoluciona, es común que se realicen múltiples modificaciones, correcciones y mejoras, por lo que resulta indispensable contar con herramientas que permitan registrar estos cambios de manera ordenada. En este contexto, Git y GitHub representan dos de las tecnologías más utilizadas a nivel mundial para implementar el control de versiones y facilitar el trabajo colaborativo.

Git es un sistema de control de versiones distribuido que permite registrar cada modificación realizada en un proyecto. Su principal función consiste en mantener un historial completo de los cambios efectuados sobre el código fuente, lo que facilita conocer qué archivos fueron modificados, quién realizó cada cambio y en qué momento se llevó a cabo. Esta información resulta de gran utilidad para identificar errores, restaurar versiones anteriores cuando sea necesario y mantener un seguimiento continuo de la evolución del proyecto. Además, Git permite trabajar de forma local, por lo que los desarrolladores pueden realizar cambios y registrar avances incluso sin conexión a Internet.

Por otra parte, GitHub funciona como una plataforma de alojamiento de repositorios basada en Git, permitiendo almacenar los proyectos en la nube y facilitar el acceso desde cualquier lugar. Su principal ventaja es que ofrece un entorno donde varios desarrolladores pueden colaborar sobre un mismo proyecto sin interferir directamente con el trabajo de los demás. Mediante el uso de repositorios remotos, ramas (branches) y solicitudes de incorporación de cambios (pull requests), GitHub hace posible organizar el desarrollo del software de una manera más segura, eficiente y profesional.

La utilización conjunta de Git y GitHub aporta importantes beneficios durante todo el ciclo de vida del desarrollo de software. Entre ellos destaca la posibilidad de mantener un respaldo constante del proyecto, reduciendo significativamente el riesgo de pérdida de información ocasionada por errores humanos o fallas en los equipos de trabajo. Asimismo, permite regresar a versiones anteriores del proyecto cuando una modificación genera problemas, evitando tener que reconstruir manualmente el código que ya había sido desarrollado.

Otro beneficio importante es la mejora en la organización del trabajo colaborativo. Cuando varias personas participan en un mismo proyecto, es frecuente que diferentes integrantes trabajen sobre distintas funcionalidades al mismo tiempo. Gracias al uso de ramas, cada desarrollador puede trabajar de manera independiente sin afectar la versión principal del proyecto. Una vez que los cambios han sido revisados y verificados, pueden integrarse al repositorio principal de forma controlada, reduciendo la posibilidad de conflictos y facilitando la revisión del código antes de su incorporación definitiva.

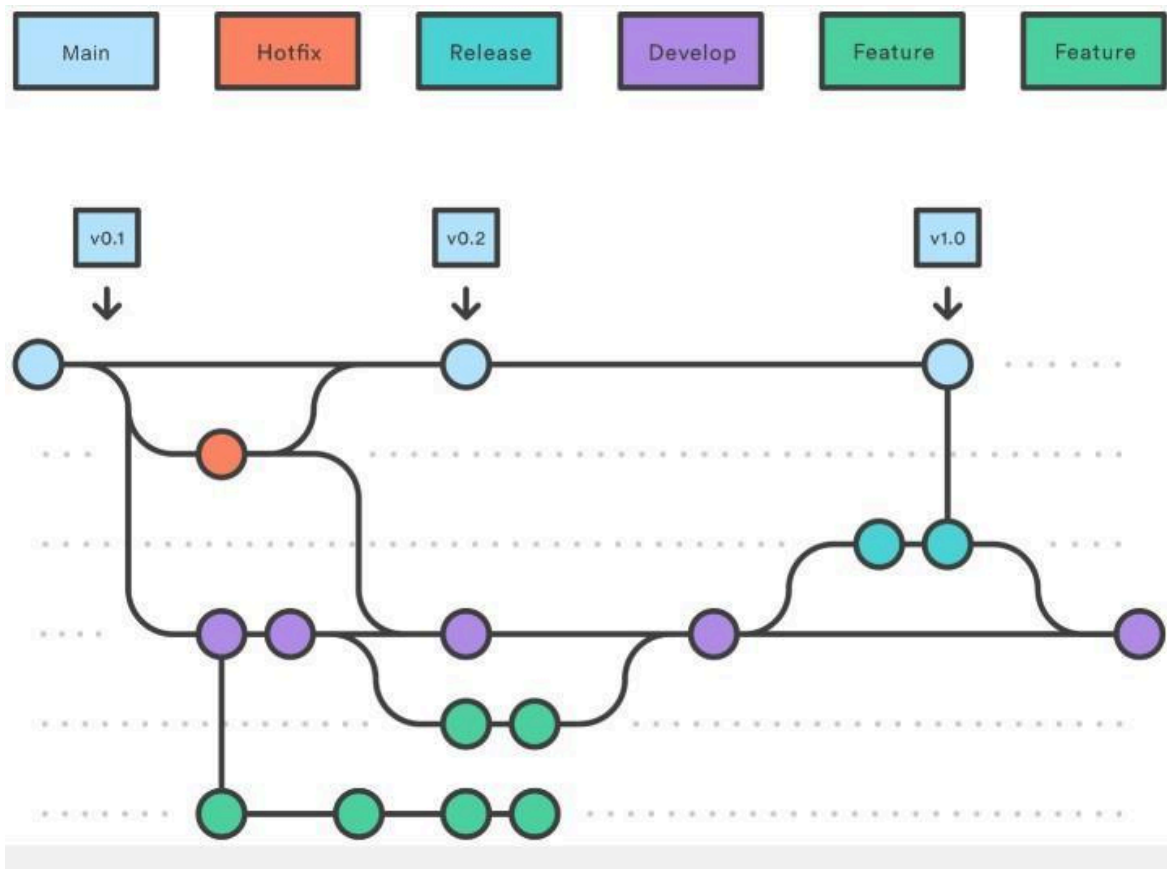
Además de favorecer la colaboración, Git y GitHub permiten llevar un mejor control del progreso del proyecto. Cada confirmación o *commit* representa un avance específico en el desarrollo, dejando evidencia del trabajo realizado y facilitando el seguimiento de cada etapa del proyecto. Esto también mejora la documentación del desarrollo, ya que el historial de cambios sirve como registro permanente de las decisiones tomadas durante la implementación del software.

Desde el punto de vista académico, el aprendizaje y uso de Git y GitHub representa una competencia indispensable para los estudiantes de desarrollo de software. Actualmente, la mayoría de las empresas dedicadas al desarrollo tecnológico utilizan sistemas de control de versiones como parte de sus procesos de trabajo, por lo que dominar estas herramientas contribuye a fortalecer las habilidades profesionales y prepara a los futuros desarrolladores para integrarse a equipos de trabajo reales bajo metodologías ágiles y prácticas modernas de ingeniería de software.

Por todo lo anterior, el uso de Git y GitHub en este proyecto se justifica como una estrategia que permite administrar eficientemente el código fuente, mantener un historial organizado de las modificaciones realizadas, facilitar el trabajo colaborativo entre los integrantes del equipo y garantizar una mayor seguridad durante el desarrollo del software. Asimismo, el repositorio creado en GitHub servirá como evidencia del proceso de versionamiento, demostrando la aplicación de buenas prácticas en la gestión del proyecto y permitiendo consultar en cualquier momento el historial de cambios y la evolución del desarrollo.

Flujo de trabajo

Git Flow



El diagrama anterior representa el modelo de trabajo **Git Flow**, uno de los flujos de control de versiones más utilizados en proyectos de desarrollo de software de mediana y gran escala. Este modelo organiza el desarrollo mediante diferentes ramas, donde cada una cumple una función específica dentro del proyecto, permitiendo mantener un desarrollo ordenado y seguro.

La rama **Main** (o *master* en versiones anteriores) representa la versión estable del proyecto. En esta rama únicamente se almacenan las versiones que ya han sido probadas y están listas para ser utilizadas por los usuarios. Cada vez que se publica una nueva versión estable, esta se identifica mediante etiquetas de versión, como se observa en el diagrama con los ejemplos **v0.1**, **v0.2** y **v1.0**.

La rama **Develop** funciona como la rama principal de desarrollo. En ella se integran todas las nuevas funcionalidades desarrolladas por el equipo antes de ser publicadas. Es considerada el punto central donde se reúne el trabajo de todos los desarrolladores para preparar futuras versiones del software.

Las ramas **Feature** se crean a partir de la rama **Develop** con el objetivo de desarrollar nuevas funcionalidades o mejoras específicas del sistema. Cada desarrollador puede trabajar de manera independiente sobre su propia rama sin

afectar el funcionamiento del proyecto principal. Una vez que la funcionalidad ha sido completada y probada, se integra nuevamente en la rama **Develop**.

Cuando el proyecto alcanza un estado suficientemente estable para preparar una nueva versión, se crea una rama **Release**. En esta etapa se realizan pruebas finales, corrección de pequeños errores, optimización del sistema y validaciones antes de publicar la versión definitiva. Una vez finalizado este proceso, la rama **Release** se fusiona tanto con **Main**, para liberar la nueva versión, como con **Develop**, para conservar todas las correcciones realizadas.

En caso de que aparezca un error crítico en una versión que ya se encuentra en producción, se utiliza una rama **Hotfix**. Esta rama nace directamente desde **Main**, ya que el objetivo es solucionar el problema con rapidez sin esperar el siguiente ciclo de desarrollo. Después de aplicar la corrección, los cambios se integran nuevamente tanto en **Main** como en **Develop**, asegurando que la solución permanezca en futuras versiones del proyecto.

El flujo mostrado en el diagrama permite observar cómo cada tipo de rama cumple una función específica dentro del desarrollo del software. Gracias a esta organización es posible trabajar simultáneamente en nuevas funcionalidades, preparar versiones para producción y corregir errores urgentes sin afectar la estabilidad del proyecto. Este modelo facilita la colaboración entre los integrantes del equipo, mejora el seguimiento de los cambios y mantiene un historial claro de la evolución del software.

Parámetros de configuración de Git y GitHub

Git

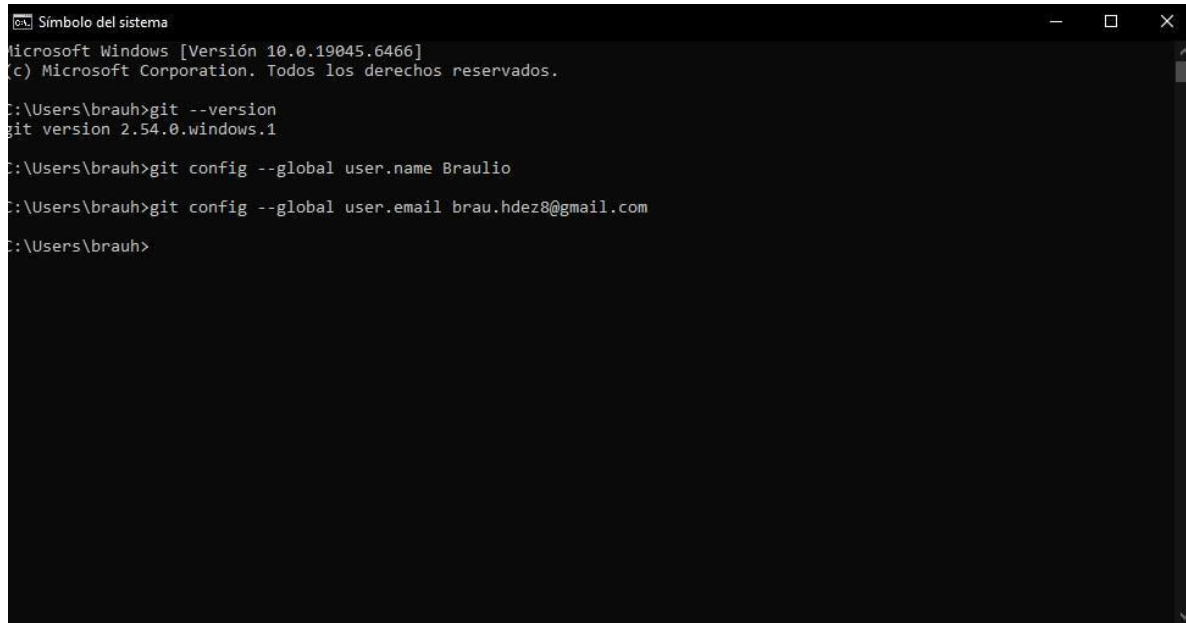
Nombre de usuario y correo electrónico: Abre el símbolo del sistema y ejecuta los siguientes comandos para configurar tu nombre de usuario y correo electrónico en Git:

```
Código git config --global user.name "Tu Nombre"
```

```
git config --global user.email
```

```
"tu_correo@ejemplo.com"
```

Reemplaza "Tu Nombre" y "tu_correo@ejemplo.com" con tu información real.



```
Símbolo del sistema
Microsoft Windows [Versión 10.0.19045.6466]
(c) Microsoft Corporation. Todos los derechos reservados.

C:\Users\brauh>git --version
git version 2.54.0.windows.1

C:\Users\brauh>git config --global user.name Braulio

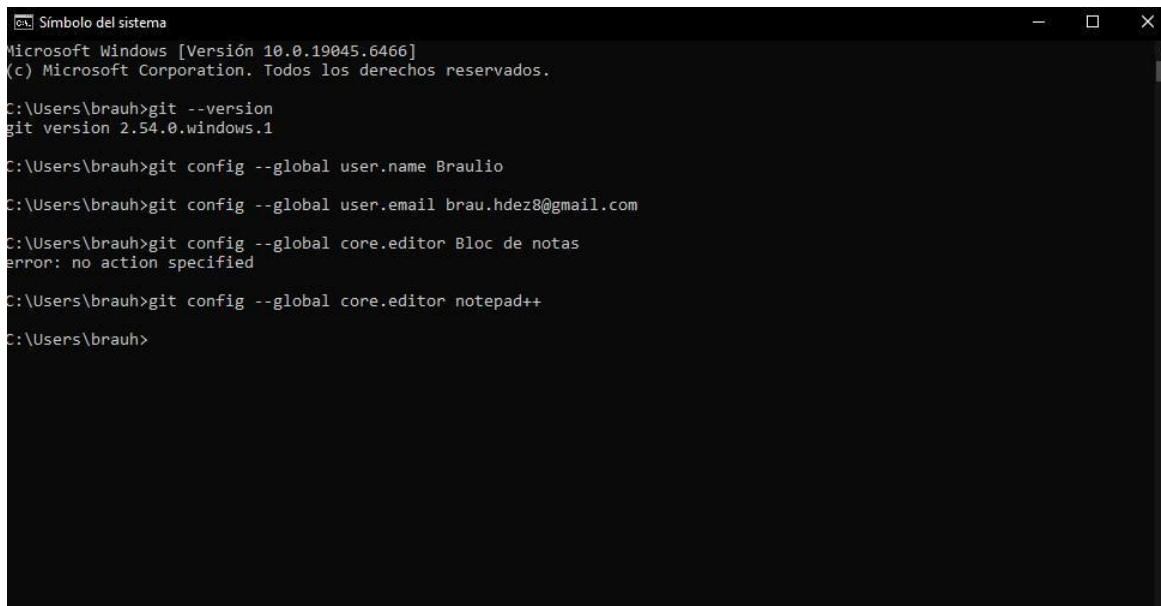
C:\Users\brauh>git config --global user.email brau.hdez8@gmail.com

C:\Users\brauh>
```

Editor de texto: Puedes configurar el editor de texto predeterminado para

Git: Código `git config --global core.editor "nuestro editor"`

Reemplaza "nuestro editor" con el nombre de tu editor de texto (por ejemplo, "notepad++", "vim", "emacs").



```
Símbolo del sistema
Microsoft Windows [Versión 10.0.19045.6466]
(c) Microsoft Corporation. Todos los derechos reservados.

C:\Users\brauh>git --version
git version 2.54.0.windows.1

C:\Users\brauh>git config --global user.name Braulio

C:\Users\brauh>git config --global user.email brau.hdez8@gmail.com

C:\Users\brauh>git config --global core.editor Bloc de notas
error: no action specified

C:\Users\brauh>git config --global core.editor notepad++

C:\Users\brauh>
```

Otros ajustes: Puedes configurar otras opciones globales de Git, como el color de la salida, el tamaño de la historia, etc.

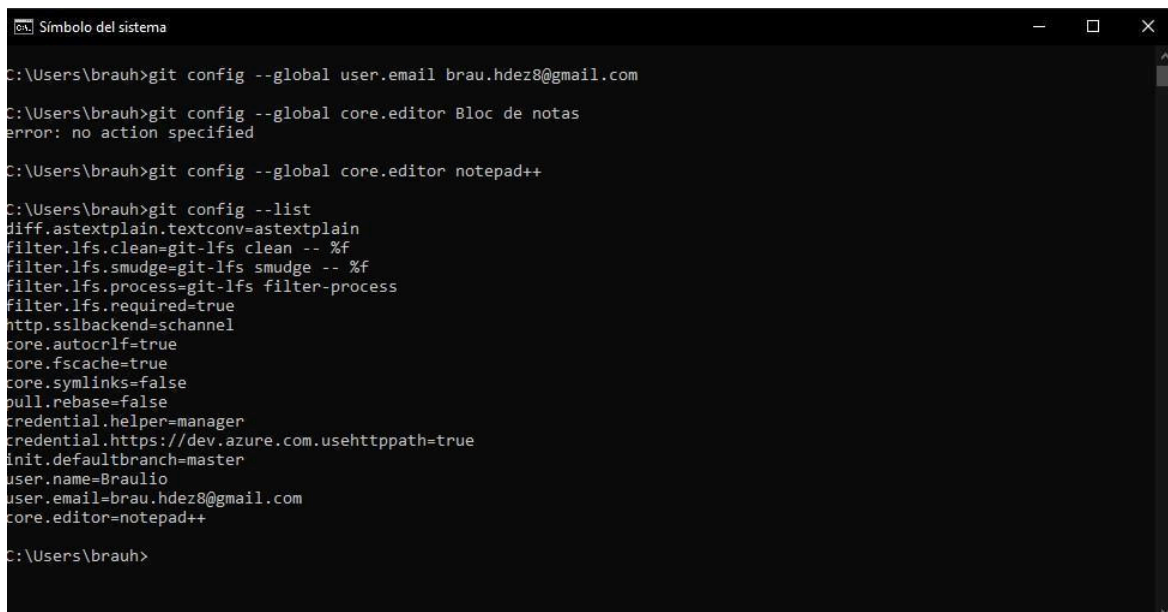
Verificación de la configuración:

Puedes verificar la configuración de Git ejecutando el comando:

Código

```
git config --list
```

Este comando mostrará todas las configuraciones globales de Git



```
C:\Users\brauh>git config --global user.email brau.hdez8@gmail.com

C:\Users\brauh>git config --global core.editor Bloc de notas
error: no action specified

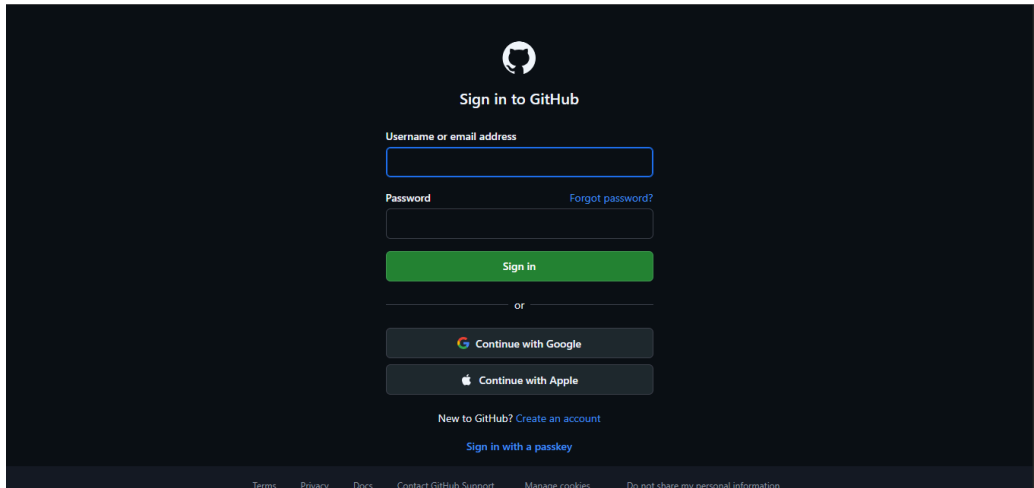
C:\Users\brauh>git config --global core.editor notepad++

C:\Users\brauh>git config --list
diff.astextplain.textconv=astextplain
filter.lfs.clean=git-lfs clean -- %f
filter.lfs.smudge=git-lfs smudge -- %f
filter.lfs.process=git-lfs filter-process
filter.lfs.required=true
http.sslbackend=schannel
core.autocrlf=true
core.fscache=true
core.symlinks=false
pull.rebase=false
credential.helper=manager
credential.https://dev.azure.com.usehttppath=true
init.defaultbranch=master
user.name=Braulio
user.email=brau.hdez8@gmail.com
core.editor=notepad++

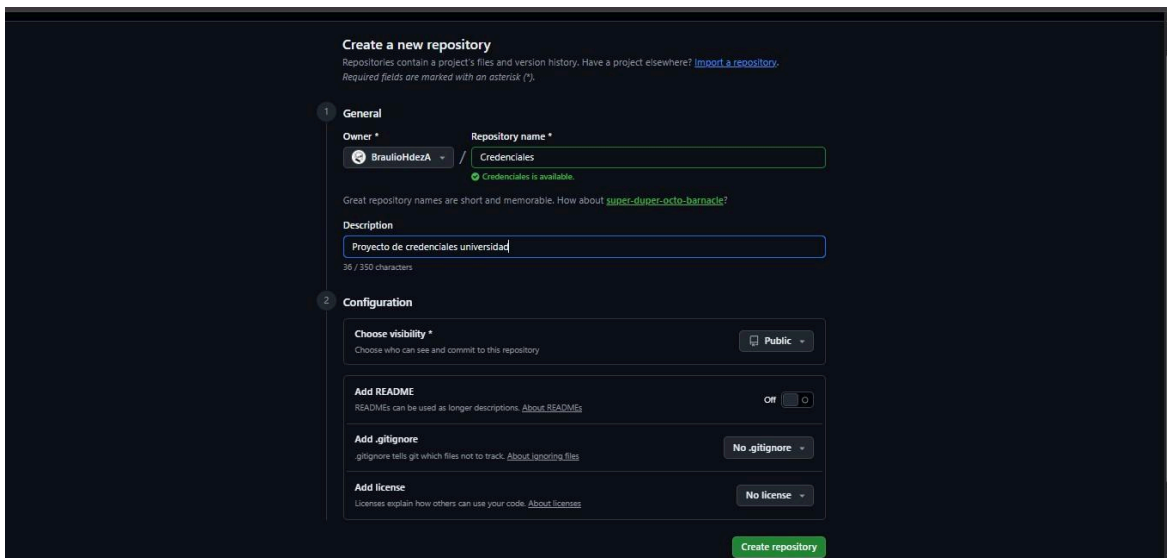
C:\Users\brauh>
```

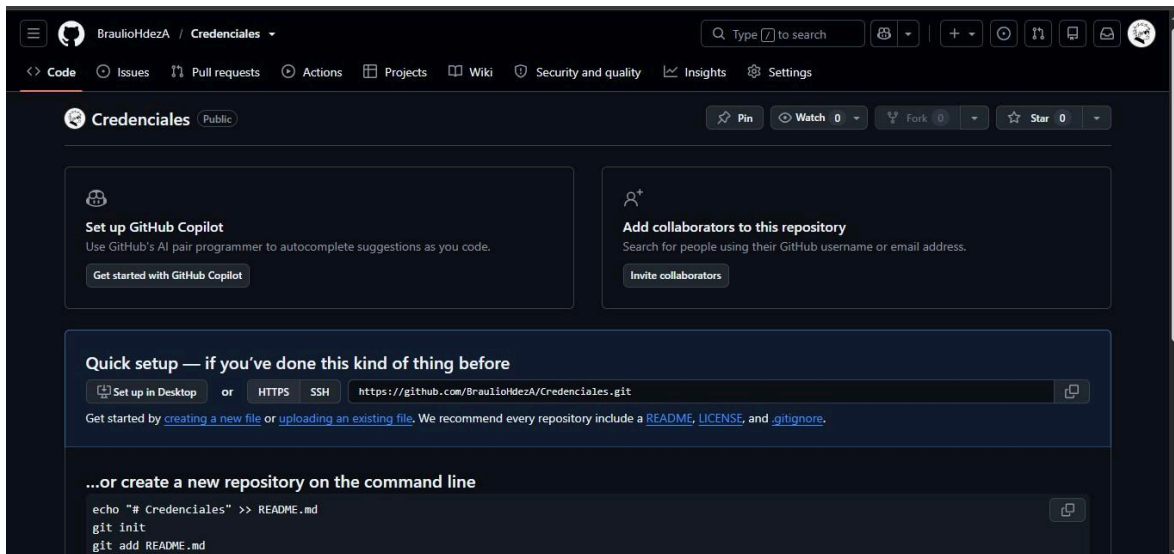
GitHub

Iniciamos sesión en GitHub



Creamos un nuevo repositorio. Le damos nombre y descripción al repositorio que tendrá el proyecto. Posteriormente copiamos el URL del proyecto y lo guardamos.





Ahora pasemos los archivos de un repositorio local al remoto.

Entra a la carpeta donde se encuentre tu proyecto a través de la terminal de git

Basch, posteriormente introduce

```
MINGW64:/c/Users/brauh/Credenciales

brauh@DESKTOP-NARUTVB MINGW64 ~ (master)
$ cd /c/Users/brauh/Credenciales

brauh@DESKTOP-NARUTVB MINGW64 ~/Credenciales (master)
$ git init
Initialized empty Git repository in C:/Users/brauh/Credenciales/.git/

brauh@DESKTOP-NARUTVB MINGW64 ~/Credenciales (master)
$ git remote add origin "https://github.com/BraulioHdezA/Credenciales.git"

brauh@DESKTOP-NARUTVB MINGW64 ~/Credenciales (master)
$
```

Veamos el status antes de hacer Push

```
brauh@DESKTOP-NARUTVB MINGW64 ~/Credenciales (master)
$ git status
On branch master

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        .gitignore
        App.js
        LICENSE
        app.json
        app/
        assets/
        package-lock.json
        package.json

nothing added to commit but untracked files present (use "git add" to track)

brauh@DESKTOP-NARUTVB MINGW64 ~/Credenciales (master)
$
```

Primero agrega todos los archivos al repositorio al área local

```

brauh@DESKTOP-NARUTVB MINGW64 ~/Credenciales (master)
$ git add .
warning: in the working copy of '.gitignore', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'App.js', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'LICENSE', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'app.json', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'package-lock.json', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'package.json', LF will be replaced by CRLF the next time Git touches it

```

Ahora chequemos el status otra vez, nos indica que ya se encuentran en el repositorio agregados. Se agregaron de forma provisional.

```

brauh@DESKTOP-NARUTVB MINGW64 ~/Credenciales (master)
$ git status
On branch master

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
    new file:   .gitignore
    new file:   App.js
    new file:   LICENSE
    new file:   app.json
    new file:   app/_layout.js
    new file:   app/accesos.js
    new file:   app/ayuda.js
    new file:   app/biblioteca.js
    new file:   app/configuracion.js
    new file:   app/credencial.js
    new file:   app/dashboard.js
    new file:   app/estado.js
    new file:   app/index.js
    new file:   app/notificaciones.js
    new file:   app/perfil.js
    new file:   app/qr.js
    new file:   app/servicios.js
    new file:   assets/android-icon-background.png
    new file:   assets/android-icon-foreground.png
    new file:   assets/android-icon-monochrome.png
    new file:   assets/favicon.png
    new file:   assets/icon.png
    new file:   assets/splash-icon.png
    new file:   package-lock.json
    new file:   package.json

```

Ahora agréguémoslos de forma definitiva utilizando Commit

```

brauh@DESKTOP-NARUTVB MINGW64 ~/Credenciales (master)
$ git commit -m "version 1"
[master (root-commit) e70d42d] version 1
25 files changed, 8622 insertions(+)
create mode 100644 .gitignore
create mode 100644 App.js
create mode 100644 LICENSE
create mode 100644 app.json
create mode 100644 app/_layout.js
create mode 100644 app/accesos.js
create mode 100644 app/ayuda.js
create mode 100644 app/biblioteca.js
create mode 100644 app/configuracion.js
create mode 100644 app/credencial.js
create mode 100644 app/dashboard.js
create mode 100644 app/estado.js
create mode 100644 app/index.js
create mode 100644 app/notificaciones.js
create mode 100644 app/perfil.js
create mode 100644 app/qr.js
create mode 100644 app/servicios.js
create mode 100644 assets/android-icon-background.png
create mode 100644 assets/android-icon-foreground.png
create mode 100644 assets/android-icon-monochrome.png
create mode 100644 assets/favicon.png
create mode 100644 assets/icon.png
create mode 100644 assets/splash-icon.png
create mode 100644 package-lock.json
create mode 100644 package.json

```

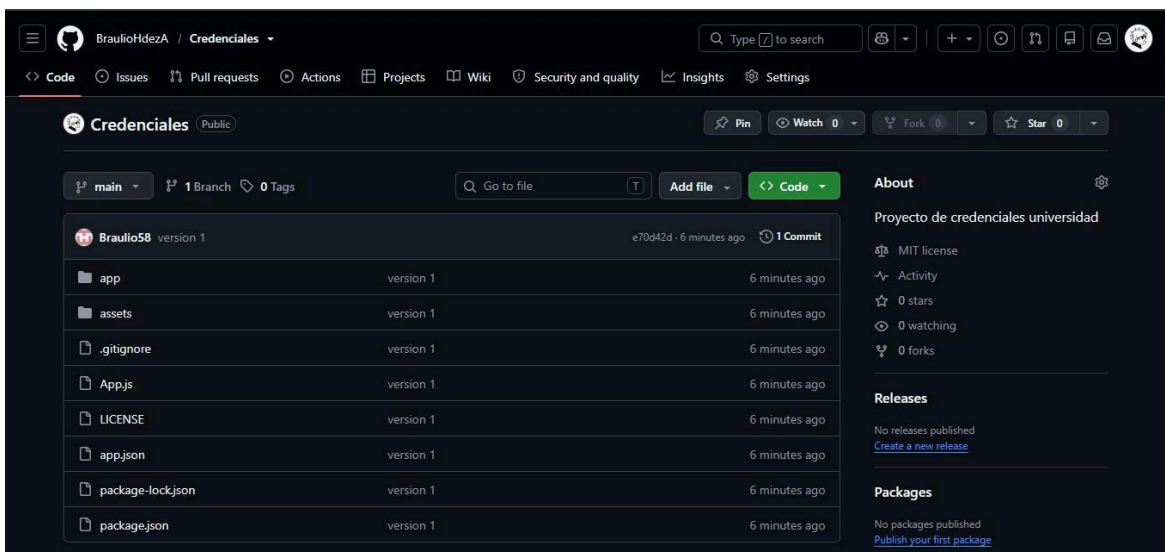
Subimos todo

```

brauh@DESKTOP-NARUTVB MINGW64 ~/Credenciales (main)
$ git push -u origin main
Enumerating objects: 28, done.
Counting objects: 100% (28/28), done.
Delta compression using up to 2 threads
Compressing objects: 100% (28/28), done.
Writing objects: 100% (28/28), 571.24 KiB | 4.84 MiB/s, done.
Total 28 (delta 4), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (4/4), done.
To https://github.com/BraulioHdezA/Credenciales.git
* [new branch]      main -> main
branch 'main' set up to track 'origin/main'.

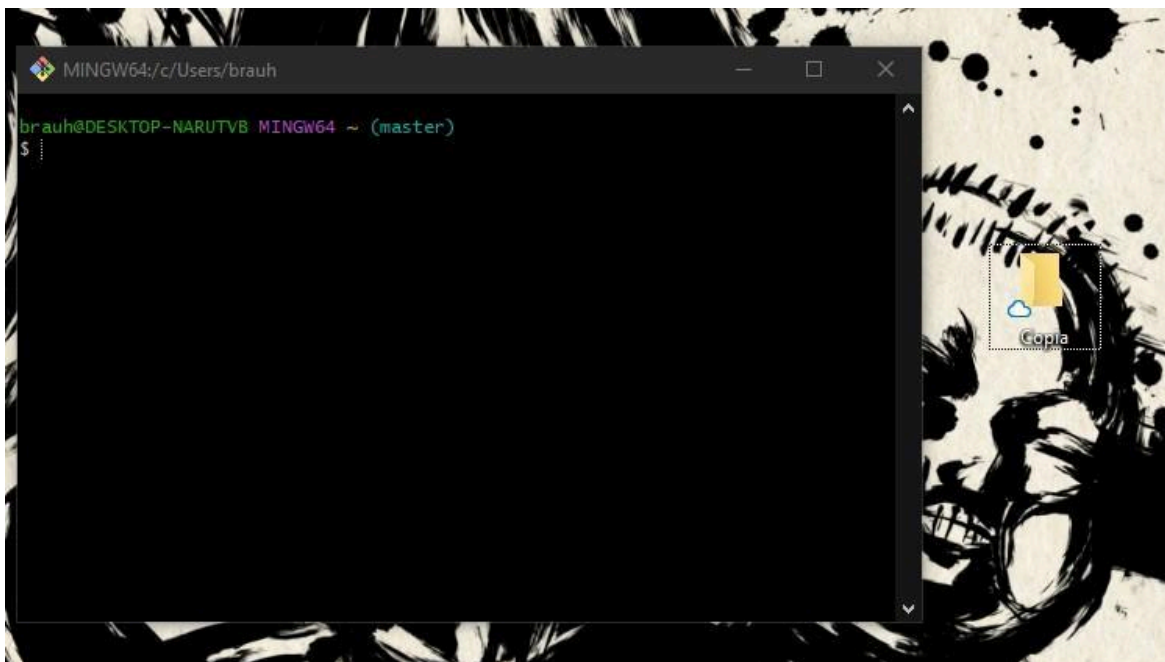
```

Ahora entra al repositorio remoto y verifica que ya se encuentra los archivos.



Clonación del repositorio en los locales

Creamos una nueva carpeta vacía y abrimos la terminal de gitbash



Ahora clonemos el proyecto que se encuentra en el repositorio remoto, para ello necesitaras el URL del proyecto. Ejecuta el siguiente comando desde la carpeta vacía y observa que en la carpeta que estaba vacía ya se encuentra clonado los archivos.


```

MINGW64:/c:/Users/brauh/OneDrive/Escritorio/Copia/Credenciales

brauh@DESKTOP-NARUTVB MINGW64 ~ (master)
$ cd C:/Users/brauh/OneDrive/Escritorio/Copia

brauh@DESKTOP-NARUTVB MINGW64 ~/OneDrive/Escritorio/Copia (master)
$ git clone https://github.com/BraulioHdezA/Credenciales.git
Cloning into 'Credenciales'...
remote: Enumerating objects: 28, done.
remote: Counting objects: 100% (28/28), done.
remote: Compressing objects: 100% (24/24), done.
remote: Total 28 (delta 4), reused 28 (delta 4), pack-reused 0 (from 0)
Receiving objects: 100% (28/28), 571.24 KiB | 1.25 MiB/s, done.
Resolving deltas: 100% (4/4), done.

brauh@DESKTOP-NARUTVB MINGW64 ~/OneDrive/Escritorio/Copia (master)
$ cd Credenciales

brauh@DESKTOP-NARUTVB MINGW64 ~/OneDrive/Escritorio/Copia/Credenciales (main)
$ ls
App.js  LICENSE  app/  app.json  assets/  package-lock.json  package.json

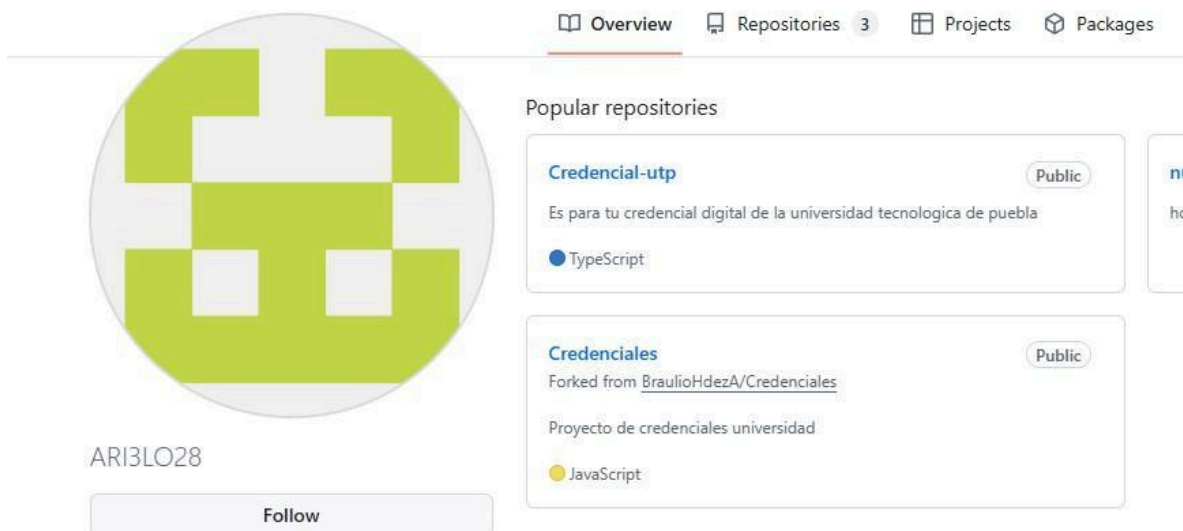
brauh@DESKTOP-NARUTVB MINGW64 ~/OneDrive/Escritorio/Copia/Credenciales (main)
$ git status
On branch main
Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean

brauh@DESKTOP-NARUTVB MINGW64 ~/OneDrive/Escritorio/Copia/Credenciales (main)
$

```

Evidencia de haber colaborado todo el equipo en el repositorio



ARI3LO28

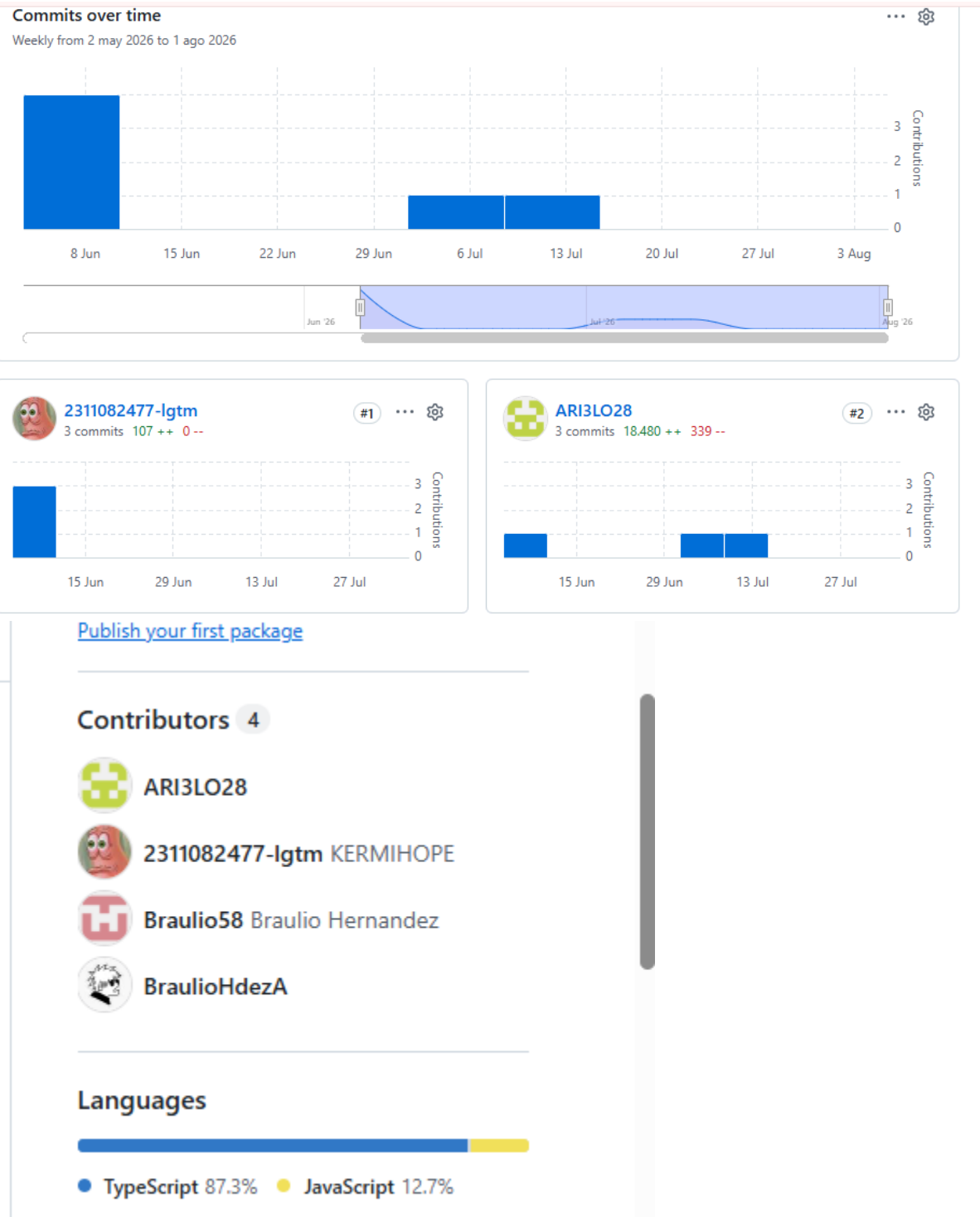
Follow

Overview Repositories 3 Projects Packages

Popular repositories

Credencial-utp Public
Es para tu credencial digital de la universidad tecnologica de puebla
TypeScript

Credenciales Public
Forked from BraulioHdezA/Credenciales
Proyecto de credenciales universidad
JavaScript



Enlace del repositorio en funcionamiento

<https://github.com/ARI3LO28/Credencial-utp.git>

Se presenta algunas ramas creadas en el repositorio

```
brauh@DESKTOP-NARUTVB MINGW64 ~/Credenciales (main)
$ git branch
* main

brauh@DESKTOP-NARUTVB MINGW64 ~/Credenciales (main)
$ git branch desarrollo

brauh@DESKTOP-NARUTVB MINGW64 ~/Credenciales (main)
$ git branch
  desarrollo
* main

brauh@DESKTOP-NARUTVB MINGW64 ~/Credenciales (main)
$ git checkout desarrollo
Switched to branch 'desarrollo'

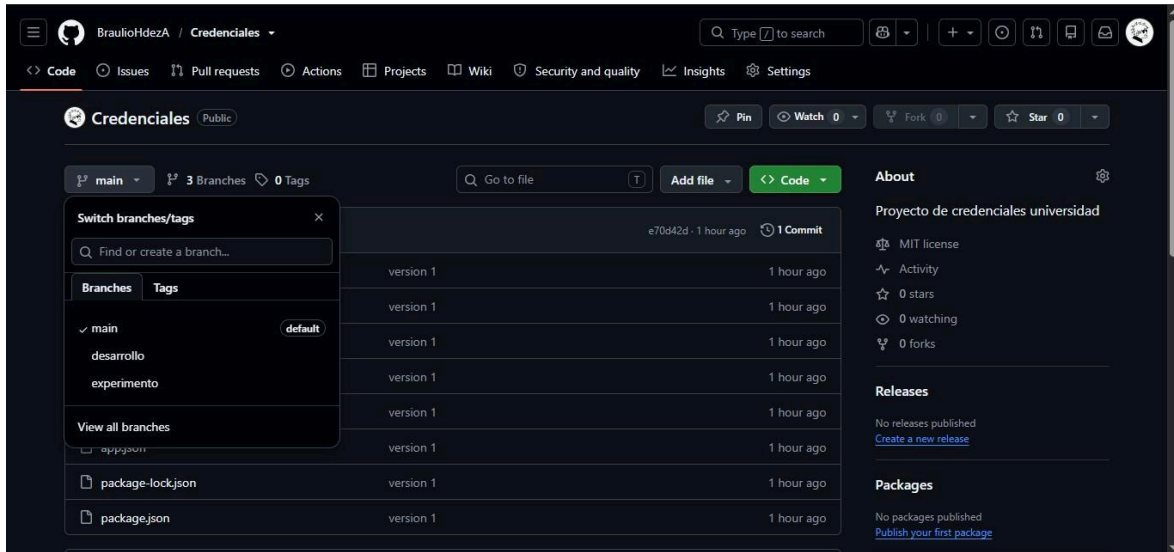
brauh@DESKTOP-NARUTVB MINGW64 ~/Credenciales (desarrollo)
$ git push -u origin desarrollo
Total 0 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
remote:
remote: Create a pull request for 'desarrollo' on GitHub by visiting:
remote:   https://github.com/BraulioHdezA/Credenciales/pull/new/desarrollo
remote:
To https://github.com/BraulioHdezA/Credenciales.git
 * [new branch]      desarrollo -> desarrollo
branch 'desarrollo' set up to track 'origin/desarrollo'.
```

```
brauh@DESKTOP-NARUTVB MINGW64 ~/Credenciales (desarrollo)
$ git branch experimento

brauh@DESKTOP-NARUTVB MINGW64 ~/Credenciales (desarrollo)
$ git checkout experimento
Switched to branch 'experimento'

brauh@DESKTOP-NARUTVB MINGW64 ~/Credenciales (experimento)
$ git push -u origin experimento
Total 0 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
remote:
remote: Create a pull request for 'experimento' on GitHub by visiting:
remote:   https://github.com/BraulioHdezA/Credenciales/pull/new/experimento
remote:
To https://github.com/BraulioHdezA/Credenciales.git
 * [new branch]      experimento -> experimento
branch 'experimento' set up to track 'origin/experimento'.

brauh@DESKTOP-NARUTVB MINGW64 ~/Credenciales (experimento)
$
```



Presentan evidencia del desarrollo de su Web}

UNIVERSIDAD TECNOLÓGICA DE PUEBLA

Alumno

PORTAL ESTUDIANTIL

Hola, Ariel

Accede a tus servicios y recursos académicos

Salir

UNIVERSIDAD TECNOLÓGICA DE PUEBLA
Credencial Digital

Ariel Abraham Cruz Sanchez

MATRÍCULA
2311082480

CARRERA
Desarrollo Web Integral

CUATRIMESTRE
9

GRUPO
D

Activo

Validación de identidad
Escanea el código QR para consultar la validación pública de esta credencial.

SERVICIOS
Accesos rápidos

Notas académicas
Calificaciones por parcial

Biblioteca
Préstamos y devoluciones

Beneficios
Descuentos vigentes

SERVICIOS

Accesos rapidos



Notas academicas

Calificaciones por parcial



Biblioteca

Prestamos y devoluciones



Beneficios

Descuentos vigentes



Actualizar foto

Foto institucional



CAMPUS

Informacion para estudiantes

CLIMA DEL CAMPUS

Lluvia



24°C

Sensacion 24°C

Humedad 37%

Lluvia 4%

Calendario academico

Suspension

2026-01-01

Año Nuevo

Dia inhabil oficial considerado para la planeacion academica.

Suspension

2026-02-02

Día de la Constitución

Dia inhabil oficial considerado para la planeacion academica.

SERVICIOS

Accesos rapidos



Notas academicas

Calificaciones por parcial



Biblioteca

Prestamos y devoluciones



Beneficios

Descuentos vigentes



Actualizar foto

Foto institucional



CAMPUS

Informacion para estudiantes

CLIMA DEL CAMPUS

Lluvia



24°C

Sensacion 24°C

Humedad 37%

Lluvia 4%

Calendario academico

Suspension

2026-01-01

Año Nuevo

Dia inhabil oficial considerado para la planeacion academica.

Suspension

2026-02-02

Día de la Constitución

Dia inhabil oficial considerado para la planeacion academica.



Validacion de identidad

Escanea el codigo QR para consultar la validacion publica de esta credencial.

Conclusión

El control de versiones representa una herramienta indispensable en el desarrollo de software, ya que permite administrar de manera organizada todos los cambios realizados durante la creación de un proyecto. A través del uso de Git y GitHub fue posible comprender la importancia de mantener un historial de modificaciones, trabajar con diferentes ramas y colaborar de forma segura sin afectar la estabilidad del proyecto principal. Asimismo, el conocimiento de flujos de trabajo como Git Flow permitió identificar la función de cada tipo de rama y la manera en que contribuyen a organizar el desarrollo, la integración de nuevas funcionalidades y la corrección de errores.

Durante la realización de esta práctica también se presentaron diversas dificultades propias del proceso de aprendizaje. Entre ellas se encontraron la configuración inicial de Git, la comprensión de conceptos como repositorios locales y remotos, la diferencia entre las ramas **Main** y **Develop**, el uso correcto de comandos como **commit**, **push**, **pull**, **merge** y **checkout**, así como la resolución de algunos errores al conectar el repositorio local con GitHub. Además, fue necesario comprender la importancia de realizar los cambios en la rama correspondiente para evitar conflictos durante la integración del código.

A pesar de estos desafíos, la práctica permitió desarrollar una mejor comprensión sobre el funcionamiento del control de versiones y la importancia de seguir un flujo de trabajo organizado. El uso de Git y GitHub no solo facilita la administración del código y el trabajo colaborativo, sino que también constituye una habilidad fundamental para el ámbito profesional, ya que son herramientas ampliamente utilizadas en la industria del desarrollo de software. La experiencia obtenida durante la creación y administración del repositorio permitió fortalecer los conocimientos adquiridos y comprender la relevancia de aplicar buenas prácticas de versionamiento en cualquier proyecto de desarrollo.

Bibliografía

- Chacon, S., & Straub, B. (2014). Pro Git (2.^a ed.). Apress. <https://git-scm.com/book/es/v2>
- Git. (s. f.). Documentación oficial de Git. <https://git-scm.com/doc>
- GitHub. (s. f.). Documentación oficial de GitHub. <https://docs.github.com/es>
- Atlassian. (s. f.). Tutoriales de Git. <https://www.atlassian.com/es/git/tutorials>
- Driessen, V. (2010). Un modelo exitoso de ramificación con Git (Git Flow). <https://nvie.com/posts/a-successful-git-branching-model/>